

I Processi

Lezione 4

Sistemi Operativi - II Modulo (Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica

(Materiale didattico per cortesia del Prof. Emiliano Casalicchio)



SAPIENZA
UNIVERSITÀ DI ROMA



Agenda

- I processi
- Comandi per la gestione dei processi



Processo

- In Unix/Linux le due entità fondamentali sono
 - **File** - descrivono/rappresentano risorse, contengono dati e programmi
 - **Processi** - permettono di elaborare dati e usare le risorse di sistema
- **Processo** = file eseguibile (programma) in esecuzione
- Per «lanciare» un processo si deve eseguire il file corrispondente
 - Digitare il nome del file da shell come un normale comando; ovvero
 - Cliccare sull'icona corrispondente (ambienti grafici)

I processi (cont.)

- Esempi di processi sono quelli creati eseguendo i comandi delle lezioni precedenti
 - `dd`, `ls`, `cat`, `cp`, `ln`, `adduser`, `groups`, `sudo`, `su`, ...
- Non tutti i comandi creano dei processi
 - `echo`
 - `cd`
 - Questi vengono eseguiti all'interno del processo di shell (la `bash` nel nostro caso) ... sì, anche la `bash` è un processo

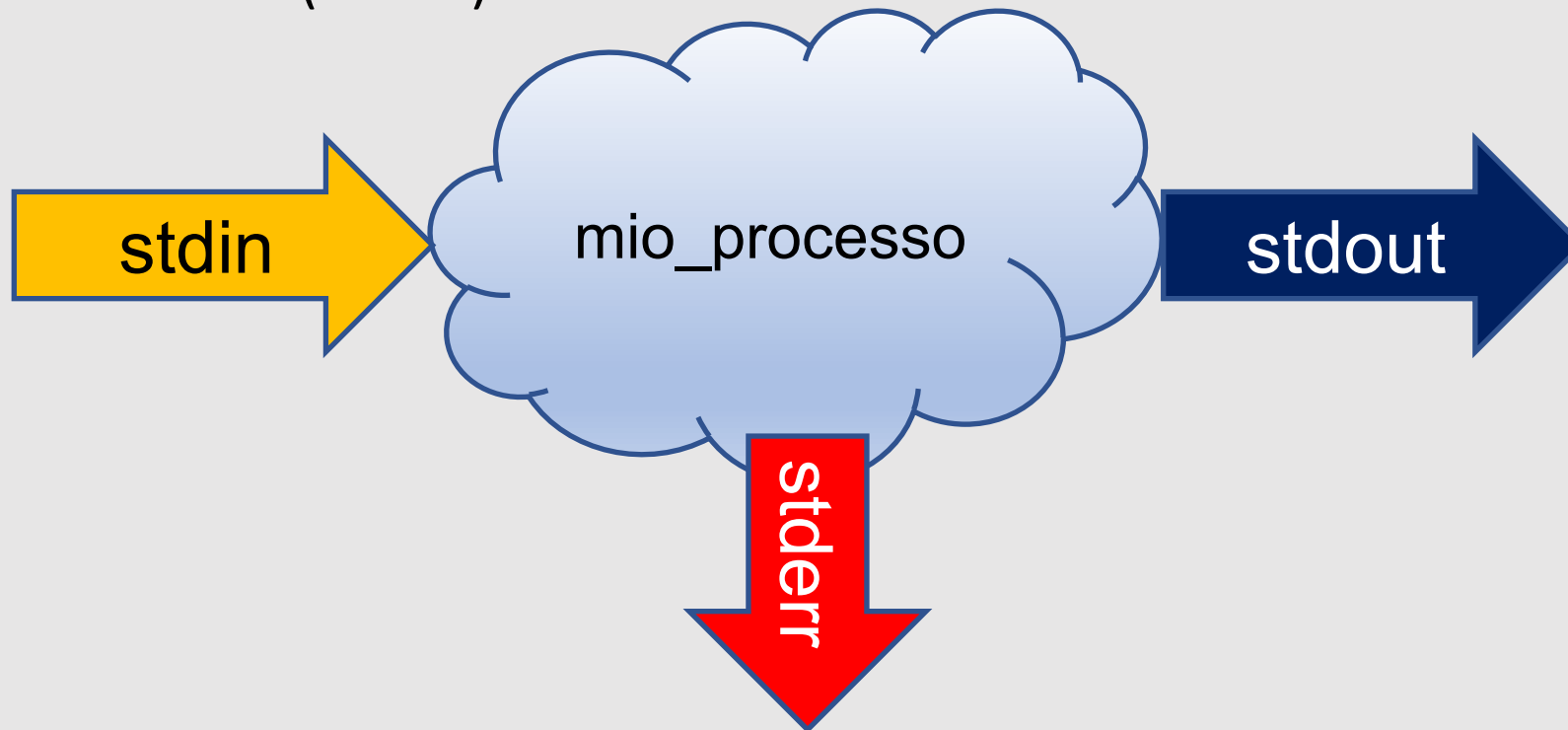


I processi (cont.)

- Un file eseguibile può essere eseguito più volte dando vita ad un nuovo processo ogni volta
- Non occorre aspettare il termine dell'esecuzione prima di lanciarlo nuovamente
 - Unix/Linux è multi-processo (multitasking)
- Ad es:
 - Aprite due shell e lanciate in entrambe il comando
`cat /dev/random`
 - Un flusso di dati casuali viene stampato sul terminale
 - CTRL+C per interrompere

Canali (dispositive/file) standard

- Ogni processo Unix/Linux ha accesso ad almeno TRE canali/file:
 - Standard **input** (*stdin*)
 - Standard **output** (*stdout*)
 - Standard **error** (*stderr*)



Canali (dispositivi) standard

- **stdin**: flusso dati in ingresso
 - default: *tastiera*
 - descrittore file: 0
- **stdout**: flusso dati in uscita
 - default: *schermo*
 - descrittore file: 1
- **stderr**: per eventuali messaggi di errore/diagnostica
 - default: *schermo*
 - descrittore file: 2
- I tre canali possono essere *ridirezionati* indipendentemente



Ridirezione dei canali standard

- I simboli `>` e `<` sono utilizzati per *ridirezionare* standard input/output/error di un comando da/su file
 - Vedi relativa sezione in `man bash`
- Ad esempio:
 - `ls > dirlist` (stdout di `ls` ridirezionato sul file `dirlist`)
 - `ls > dirlist 2>&1` (stdout di `ls` ridirezionato su `dirlist`, stderr (2) ridirezionato su stdout (1). Quindi sia standard error sia standard output sono ridirezionati su `dirlist`.)
 - `ls 2>&1 > dirlist` (standard error (2) ridirezionato su stdout (1). Standard output di `ls` ridirezionato su `dirlist` dopo che stderr è stato ridirezionato e quindi `dirlist` contiene solo lo standard output di `ls`.) [La ridirezione non è transitiva.]

Ridirezione dei canali standard

- Provate:

1. `cat` (cat senza argomenti copia stdin su stdout, *linea per linea*. Il tasto/carattere invio crea una nuova linea; CTRL+d è il carattere di fine input.)
2. `cat > fileditesto` (stdout di `ls` ridirezionato su `dirlist`, stderr (2) ridirezionato su stdout (1). Quindi sia standard error sia standard output sono ridirezionati su `dirlist`.)
3. `cat < fileditesto` (fate l'esempio 2. prima di questo)

Rappresentazione dei processi

- Process Identifier (PID)
- Process Control Block (PCB)
- Sei aree di memoria



PID - Process IDentifier

- E' un identificatore (numero intero) univoco di un processo
- In un dato istante, non ci possono essere 2 processi con lo stesso PID
- Una volta che un processo è terminato, il suo PID viene liberato, e potrebbe essere prima o poi riusato per un altro processo
- Alcuni sistemi, Linux incluso, assegnano PID *casuali* ai processi per incrementare la sicurezza. [Ad esempio: in un sistema parzialmente compromesso, un attacco al processo web server dovrebbe prima di tutto scoprire il PID di tale processo.]
- Nell'ultima decade aumento dell'uso della *casualità (randomness)* nei sistemi operativi per ridurre (*mitigate*) la superficie d'attacco



Process Control Block

PCB è unico per ogni processo

- **PID:** Process Identifier
- **PPID:** Parent Process Identifier
- **Real UID:** Real User Identifier
- **Real GID:** Real Group ID
- **Effective UID:** Effective User Identifier (UID assunto dal processo in esecuzione)
- **Effective GID:** Effective Group ID (come sopra per GID)
- **Saved UID:** Saved User Identifier (UID avuto prima dell'esecuzione del SetUID)
- **Saved GID:** Saved Group Identifier (come sopra per GID)
- **Current Working Directory:** directory di lavoro corrente
- **Umask:** file mode creation mask
- **Nice:** priorità statica del processo



Saved UID: Saved User Identifier

- Permette ad un processo che gira come root (ad es. via setuid sull'eseguibile) di fare operazioni come utente non-privilegiato e poi ritornare ai privilegi di root:
 - Ad es: il processo crea dei file che apparterranno all'utente non-privilegiato (altrimenti apparterebbero a root)
 - E' buona regola per la sicurezza di usare i privilegi di root solo quando richiesti dal kernel

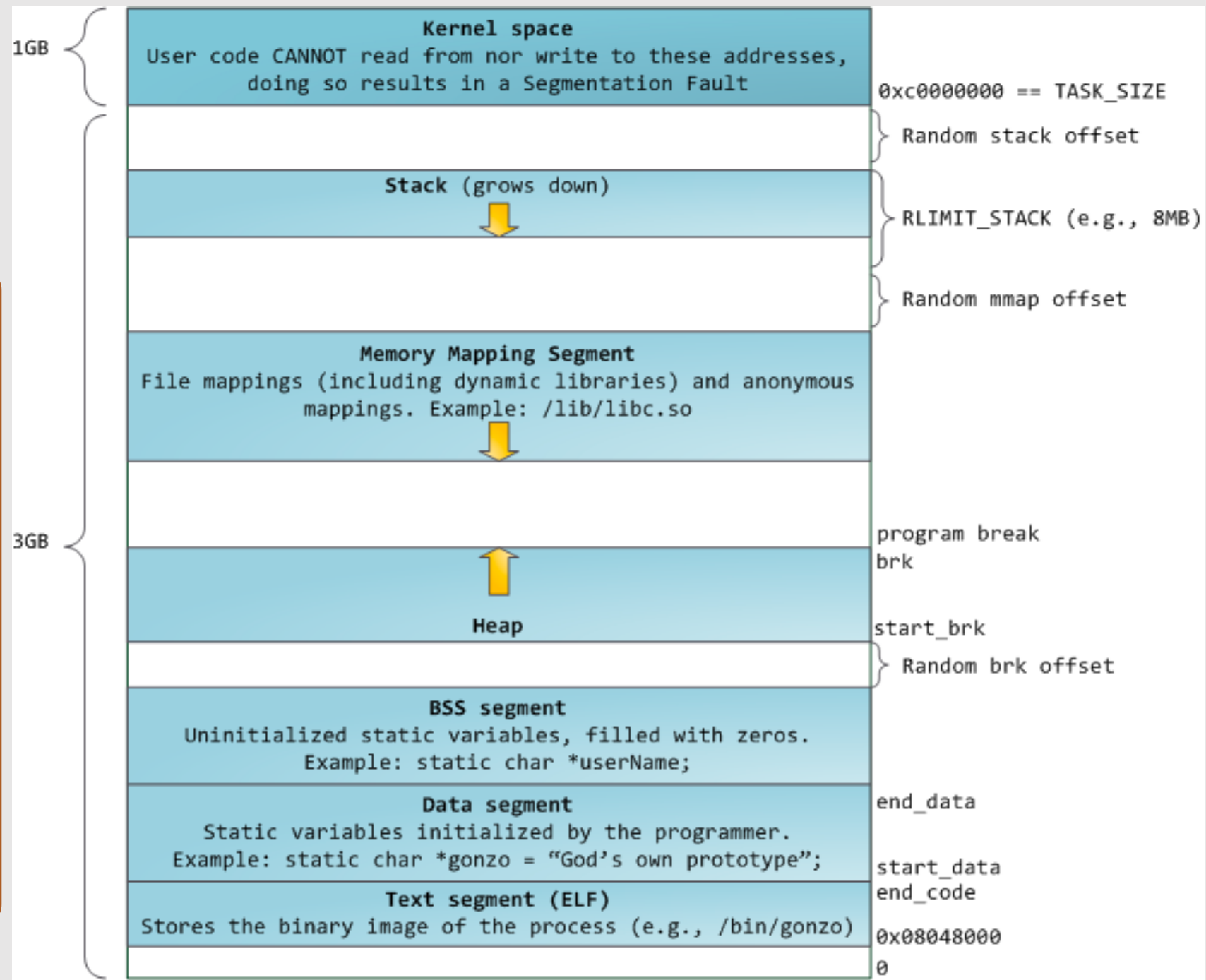
Aree di memoria

- **Text Segment:** istruzioni da eseguire (in linguaggio macchina)
- **Data Segment:** dati statici (ovvero, variabili globali e variabili locali static) inizializzati e alcune costanti di ambiente
- **BSS:** dati statici non inizializzati (sta per *block started from symbol*); la distinzione dal segmento dati si fa per motivi di realizzazione hardware
- **Heap:** dati dinamici (allocati con `malloc` e simili)
- **Stack:** per le chiamate a funzioni, con i corrispondenti dati dinamici (parametri, variabili locali non static)
- **Memory Mapping Segment:** tutto ciò che riguarda librerie esterne dinamiche usate dal processo, nonché estensione dello heap in alcuni casi

Aree di memoria

Aree di memoria potrebbero essere condivise:

- il text segment tra più istanze dello stesso processo
- due processi potrebbero avere lo stesso BSS o Data segment o MMS
- Lo stack non è mai condiviso



Stati di un processo

- **Running (R):** in esecuzione su un processore
- **Runnable (R):** pronto per essere eseguito (non è in attesa di alcun evento); in attesa che lo scheduler lo (ri)selezioni per l'esecuzione
- **(Interruptible) Sleep (S):** è in attesa di un qualche evento (ad esempio, lettura di blocchi dal disco), e non può quindi essere scelto dallo scheduler
- **Zombie (Z):** il processo è terminato e le sue 6 aree di memoria non sono più in memoria; tuttavia, il suo PCB viene ancora mantenuto dal kernel perché il processo padre non ha ancora richiesto il suo "exit status" (ritorneremo su questo punto)
- **Stopped (T):** caso particolare di sleep: avendo ricevuto un segnale STOP, è in attesa di un segnale CONT
- **Traced (t):** in esecuzione di debug, oppure in generale in attesa di un segnale (altro caso particolare di sleep; vedremo più avanti)
- **Uninterruptible sleep (D):** come sleep, ma tipicamente sta facendo operazioni di I/O su dischi lenti e non può essere interrotto o ucciso



Modalità di Esecuzione dei processi

- *Foreground* («in primo piano»):
 - praticamente tutti i casi visti finora
 - il comando può leggere l'input da tastiera e scrivere su schermo;
 - finché non termina, il prompt non viene restituito e non si possono lanciare altri comandi (o *lavori, job...*) alla shell
- *Background* («in secondo piano», o «in sottofondo») :
 - il comando non può leggere l'input da tastiera ma può scrivere su schermo
 - il prompt viene immediatamente restituito
 - mentre il lavoro viene eseguito in background, si possono da subito dare altri comandi alla shell

Esecuzione in background

- & (*ampersand*)
- Non disponibile in tutte le shell
 - ma in bash si!!

Numero del job (partono da 1) e PID

```
studente@debian9:~$  
studente@debian9:~$ sleep 15s  
^C  
studente@debian9:~$ sleep 15s &  
[1] 2634  
studente@debian9:~$  
[1]+  Completato sleep 15s  
studente@debian9:~$
```

- `sleep` mette in pausa per un tempo specificato nell'argomento (si veda il man)
- Provate a lanciare 3 volte di seguito `sleep 20s &`
 - Cosa succede al numero del job?

Lista di job

- Per vedere la lista di lavori in esecuzione

- `jobs [-l] [-p]`

```
[3]+ Completato sleep 15s
studente@debian9:~$ sleep 30s &
[1] 2767
studente@debian9:~$ sleep 30s &
[2] 2769
studente@debian9:~$ jobs -l
[1]- 2767 In esecuzione sleep 30s &
[2]+ 2769 In esecuzione sleep 30s &
studente@debian9:~$ jobs -p
2767
2769
studente@debian9:~$
```

- Se un lavoro è composto da più compiti (*pipelining*)

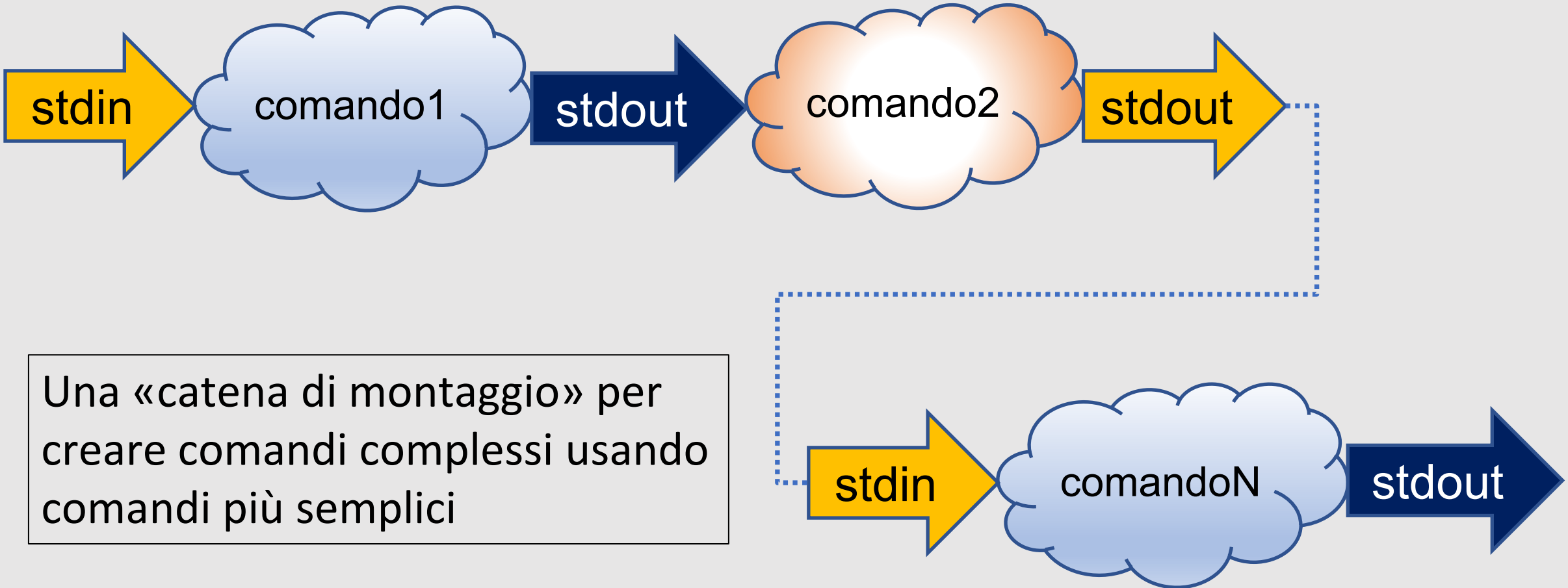
```
studente@debian9:~$
studente@debian9:~$ sleep 30s | sleep 20s | sleep 10 &
[1] 2859
studente@debian9:~$ jobs -l
[1]+ 2857 In esecuzione sleep 30s
      2858 | sleep 20s
      2859 | sleep 10 &
studente@debian9:~$ jobs -l
```

Comandi `bg` e `fg`

- Comandi di `bash` (vedi `man bash`)
- `bg` porta un processo in background
 - lancio processo; lo interrompo con `CTRL+z`; lo risveglio con `bg`
- `fg %n` dove `%n` è il numero del job, lo riporta in foreground
- `bg %n` porta in background il processo `%n`

```
studente@debian9:~$ sleep 60s
^Z
[1]+  Fermato                  sleep 60s
studente@debian9:~$ bg
[1]+  sleep 60s &
studente@debian9:~$ sleep 60s
^Z
[2]+  Fermato                  sleep 60s
studente@debian9:~$ bg
[2]+  sleep 60s &
studente@debian9:~$ fg 1
sleep 60s
```

Pipelining dei comandi



Pipelining dei comandi

- Per eseguire un job composto da più comandi

```
comando1 | comando2 | ... comando n
```

- Lo standard output di un comando i diventa l'input del comando $i+1$
- Se uso `| &` è lo standard error che viene ridirezionato sullo standard input del comando successivo
- Il comando $i+1$ non deve necessariamente usare l'output/stderr del comando i

Esercizio 1

- Lanciare 3 comandi ed interromperli ogni volta con CTRL+z, ad esempio
 - `sleep 60s CTRL+z`
 - `ls -R / CTRL+z`
 - `sleep 10s CTRL+z`
- Come faccio a riportare in esecuzione in background `ls` ?
 - prima cercate di capire come si fa e poi fatelo per evitare che `ls` termini!!!!
- Perché continuo a vedere l'output di `ls` dopo averlo riportato in esecuzione?
- *Ovviamente potete scegliere di eseguire 3 comandi a piacere*

Comandi `bg` e `fg` (cont.)

- Si possono identificare job anche con:
 - `%prefix`: dove `prefix` è la parte iniziale del comando del job desiderato;
 - `%+` oppure `%%` : l'ultimo job mandato
 - `%-`: il penultimo job mandato



Comando `ps` [opzioni] [pid...]

- Mostra le informazioni dei processi in esecuzione
- Legge le informazioni dai file virtuali in `/proc`
- `ps` senza argomenti mostra i processi dell'utente attuale lanciati dalla shell corrente
- Per ogni processo mostra PID, TTY, TIME e CMD

```
studente@debian9: ~  
File Modifica Schede Aiuto  
niente /dev/null  
[3]+ Ucciso yes niente /dev/null  
studente@debian9:~$  
studente@debian9:~$ ps  
  PID TTY          TIME CMD  
 1970 pts/0    00:00:00 bash  
 3352 pts/0    00:00:00 ps  
studente@debian9:~$ sleep 60 &  
[1] 3357  
studente@debian9:~$ sleep 80 &  
[2] 3358  
studente@debian9:~$ yes niente > /dev/null &  
[3] 3360  
studente@debian9:~$ yes altroniente > /dev/null &  
[4] 3362  
studente@debian9:~$ ps  
  PID TTY          TIME CMD  
 1970 pts/0    00:00:00 bash  
 3357 pts/0    00:00:00 sleep  
 3358 pts/0    00:00:00 sleep  
 3360 pts/0    00:00:05 yes  
 3362 pts/0    00:00:00 yes  
 3364 pts/0    00:00:00 ps  
studente@debian9:~$
```

```
studente@debian9:~$ pc  
bash: pc: comando non trovato  
studente@debian9:~$ ps  
  PID TTY          TIME CMD  
 3289 pts/1    00:00:00 bash  
 3368 pts/1    00:00:00 ps  
studente@debian9:~$
```

Opzioni `ps` `[-e]` `[-u]` `[-p]`

- `-e`: tutti i processi di tutti gli utenti lanciati da tutte le shell o al boot (figli del processo 0)
- `-u {utente, }`: tutti i processi degli utenti nella lista
 - Es: `ps -u studente`
- `-p {pid, }`: tutti i processi con i PID nella lista

Opzioni ps [-f] [-l] [-o]

```
studente@debian9:~$ ps
  PID TTY          TIME CMD
 1970 pts/0        00:00:00 bash
 3582 pts/0        00:00:00 ps
studente@debian9:~$ ps -f
UID          PID  PPID  C  STIME TTY          TIME CMD
studente    1970   1968  0  mar20 pts/0        00:00:00 bash
studente    3584   1970  0  00:27 pts/0        00:00:00 ps -f
studente@debian9:~$ ps -l
 F S      UID      PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
 0 S    1000    1970   1968  0  80   0  -  5501  -      pts/0        00:00:00 bash
 0 R    1000    3586   1970  0  80   0  -  7467  -      pts/0        00:00:00 ps
studente@debian9:~$
```

- `-o {field, }` per scegliere i campi da visualizzare; nel man ci sono gli acronimi

Significato dei campi mostrati da `ps`

- `PPID` parent pid, pid del processo che ha creato questo processo
- `C` parte intera della percentuale di uso della CPU
- `STIME` (○ `START`) l'ora in cui è stato invocato il comando, oppure la data, se è stato fatto partire da più di un giorno
- `TIME` tempo di CPU usato finora
- `CMD` comando con argomenti
- `F` flags associati al processo: 1 il processo è stato “biforcato”, ma ancora non eseguito; 4, ha usato privilegi da superutente; 5, entrambi i precedenti; 0, nessuno dei precedenti
 - `-y -l` elimina questo campo che è poco utile

Significato dei campi mostrati da `ps` (cont.)

- `S` stato (modalità) del processo in una sola lettera
 - vedere slide 13 (le lettere corrispondenti sono tra parentesi)
- `UID` utente che ha lanciato il processo (se `SetUID` presente potrebbe non essere chi ha dato il comando)
- `PRI` attuale priorità del processo (più il numero è alto, minore è la priorità)
- `NI` valore di nice, da aggiungere alla priorità (vedere più avanti)
- `ADDR` indirizzo in memoria del processo, ma è mostrato (senza valore) solo per compatibilità all'indietro
 - `-y -l` toglie questo campo e lo sostituisce con `RSS` (*resident set size*) dimensione del processo in memoria principale in KB (non tiene conto delle pagine su disco)

Significato dei campi mostrati da `ps` (cont.)

- `SZ` - dimensione totale attuale del processo in numero di pagine (tutte le 6 aree di memoria del processo) RAM + disco (memoria virtuale)
- `WCHAN` - se il processo è in attesa di un qualche segnale o comunque in sleep, qui c'è la funzione del kernel all'interno della quale si è fermato

Esercizio 2

- Copiare `/bin/sleep` nella directory corrente, cambiargli proprietario e gruppo in root e mettergli il setuid (occorre `sudo...`);
- Lanciare `sleep` che avete copiato nella `cwd` in background e farsi stampare sia l'utente reale sia quello effettivo del processo
- Analizzando il man di `ps` potete scoprire tutti i campi che possono essere visualizzati

Comando `top [-b] [-n num] [-p {pid,}]`

- Un `ps` interattivo
 - `-b` non accetta più comandi interattivi, ma aggiorna i dati ogni pochi secondi
 - `-n num` fa solo `num` aggiornamenti
 - `-p` come in `ps`
- Una volta aperto in modo interattivo, basta premere `?` per avere la lista dei comandi accettati (tipicamente, sono singole lettere)

Esercizio 3

Fare in modo che `ps` abbia lo stesso output di `top -b -n1`



```
kill [-l [signal]] [-signal] [pid...]
```

- Invia segnali ad un processo (non solo la terminazione)
- `-l` mostra la lista dei segnali; un segnale `signal` è identificato dal numero oppure dal nome con `SIG` o senza `SIG`. Ad es:

```
kill -9 pid = kill -s SIGKILL pid = kill -s KILL pid
```

```
studente@debian9:~$ kill -l
```

1) SIGHUP	2) SIGINT	3) SIGQUIT	4) SIGILL	5) SIGTRAP
6) SIGABRT	7) SIGBUS	8) SIGFPE	9) SIGKILL	10) SIGUSR1
11) SIGSEGV	12) SIGUSR2	13) SIGPIPE	14) SIGALRM	15) SIGTERM
16) SIGSTKFLT	17) SIGCHLD	18) SIGCONT	19) SIGSTOP	20) SIGTSTP
21) SIGTTIN	22) SIGTTOU	23) SIGURG	24) SIGXCPU	25) SIGXFSZ
26) SIGVTALRM	27) SIGPROF	28) SIGWINCH	29) SIGIO	30) SIGPWR
31) SIGSYS	34) SIGRTMIN	35) SIGRTMIN+1	36) SIGRTMIN+2	37) SIGRTMIN+3
38) SIGRTMIN+4	39) SIGRTMIN+5	40) SIGRTMIN+6	41) SIGRTMIN+7	42) SIGRTMIN+8
43) SIGRTMIN+9	44) SIGRTMIN+10	45) SIGRTMIN+11	46) SIGRTMIN+12	47) SIGRTMIN+13
48) SIGRTMIN+14	49) SIGRTMIN+15	50) SIGRTMAX-14	51) SIGRTMAX-13	52) SIGRTMAX-12
53) SIGRTMAX-11	54) SIGRTMAX-10	55) SIGRTMAX-9	56) SIGRTMAX-8	57) SIGRTMAX-7

kill (cont.)

- I segnali verranno presi in considerazione solo se il *real user* del processo è lo stesso che invia il segnale (oppure, se lo invia un superuser)
- Un processo che riceve un segnale fa o un'azione predefinita (vedere `man 7 signal`) o un'azione personalizzata (su questo torneremo in un'altra lezione)

Alcuni segnali

- SIGSTOP, SIGSTP sospensione processo
- SIGCONT per la continuazione di processi bloccati
- SIGKILL, SIGINT per la terminazione dei processi
- *NOTA:*
 - CTRL+z invia un SIGSTOP
 - se si manda SIGSTOP a sleep il comando continua a contare i secondi
 - CTRL+c invia un SIGINT
 - bg invia SIGCONT al job indicato
 - Con kill si può usare la notazione con % (usata in bg e fg) per indicare i job destinatari del messaggio

- **Esercizio 4** - lanciare un processo in foreground, poi (usando un'altra shell...) simulare la pressione di `CTRL+z` e di `CTRL+c`, nonché il comando `bg` (per sapere il PID di un processo in esecuzione su un'altra shell, si può usare il nome del comando, vedere l'opzione `-C` di `ps`)
- **Esercizio 5** - verificare di avere almeno 1 GB di spazio, e creare con `dd` un file di 1 GB (tutto di zeri). Subito dopo aver lanciato il comando, mandare al processo di `dd` il segnale `USR1`. Cosa succede? Come lo possiamo interpretare? [Potete fare questo esercizio anche con il comando `sleep`.]

SIGUSR1 e SIGUSR2

- I segnali SIGUSR1 e SIGUSR2 sono impostati per essere usati dall'utente per le proprie necessità.
- Essi consentono una semplice forma di comunicazione tra processi
- Ad es: In un programma P1 si può definire un gestore del segnale (*signal handler*) per SIGUSR1(o 2). Se un programma P2 invia un SIGUSR1 (o 2) a P1, questo eseguirà il codice del gestore del segnale
 - Vedremo esempi quando faremo programmazione di sistema



`nice [-n num] [comando]`

- `nice` senza opzioni dice quant'è il *niceness* di partenza
- il *niceness*
 - può essere pensato come una addizione sulla priorità: se positivo, ne aumenta il valore (quindi la priorità decresce), altrimenti ne diminuisce il valore (la priorità cresce)
 - va da -19 a +20, con default a 0
- `nice [-n num] comando` lancia comando (che potrebbe avere delle sue opzioni, più o meno come con `sudo`) con *niceness* `num` (0 se non dato)



```
renice priority {pid }
```

- Interviene su processi già in esecuzione infatti, richiede dei PID (ci sarebbero anche altre opzioni per fargli avere effetti sui processi di un dato utente/gruppo, ma soprassediamo)



`strace [-p pid] [comando]`

- Lancia `comando` visualizzando tutte le sue chiamate di sistema
- Oppure, visualizza le chiamate di sistema del processo `pid`
- `-o filename` ridireziona l'output su di un file
- Ci sarà utile per il *debugging* («disinfestazione») di programmi che utilizzano chiamate di sistema

Domande?

